

Dictionnaires

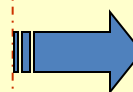
- Les types composites (chaînes de caractères, listes et tuples) considérés jusqu'à maintenant étaient tous des séquences, i.e. des suites ordonnées d'éléments.
- On peut accéder à un élément d'une séquence à partir de sa position.
- Un dictionnaire ressemble à une liste et est modifiable mais n'est pas une séquence car les éléments enregistrés ne sont pas disposés dans un ordre immuable.
- On peut accéder à un élément d'un dictionnaire à partir d'une clé. Cette clé peut être une chaîne, un nombre ou même d'un type composite sous certaines conditions. On ne peut pas modifier les clés d'un dictionnaire.
- Puisque le type dictionnaire est un type modifiable, nous pouvons commencer par créer un dictionnaire vide noté {}, puis le remplir par la suite.

```
>>> traduction = {}  
>>> print(traduction)
```



dictionnaire vide

```
{}  
>>> traduction["mouse"] = "souris"  
>>> traduction["keyboard"] = "clavier"  
>>> print(traduction)  
{'mouse': 'souris', 'keyboard': 'clavier'}
```



insertion d'éléments à l'aide de paires clé-valeur



Le dictionnaire est entouré de {}.

Les dictionnaires

- On peut aussi créer un dictionnaire comme suit :

```
>>> D = {"kh" : "khalil", "ch" : "chrit"}
>>> F, G = {}, {5: "Mauve", 2: "Rouge"}
>>> print (D, F, G)
{"kh" : "khalil", "ch" : "chrit"} {} {2: 'Rouge', 5: 'Mauve'}
```

- L'ordre dans lequel les éléments apparaissent à l'affichage du dictionnaire ne correspond pas nécessairement à celui dans lequel nous les avons insérés. Cela n'a pas d'importance car nous utilisons les clés pour accéder à un élément.

```
>>> Couleur = {1: "Rouge", 2: "Vert", 3: "Bleu"}
>>> print(Couleur)
{1: 'Rouge', 2: 'Vert', 3: 'Bleu'}
>>> Couleur = {'1': "Rouge", '2': "Vert", '3': "Bleu"}
>>> print(Couleur)
{'1': 'Rouge', '3': 'Bleu', '2': 'Vert'}
```

- Illustrons maintenant la variété de types de clés qu'on peut utiliser.

```
>>> D = {1: "un", '1': "valeur_unitaire", 1.1: "valeur_decimale"}
>>> print(D)
{'1': 'valeur_unitaire', 1: 'un', 1.1000000000000001: 'valeur_decimale'}
>>> D[1.1]
'valeur_decimale'
```

Les dictionnaires

- Nous pouvons utiliser en guise de clés n'importe quel type de donnée non modifiable : des entiers, des réels, des chaînes de caractères et même des tuples.

```
>>> sudoku = {}
>>> sudoku[(0, 1)] = 9
>>> sudoku[(0, 4)] = 5
>>> sudoku[(0, 7)] = 3
>>> print(sudoku)
{(0, 1): 9, (0, 7): 3, (0, 4): 5}
>>> print(sudoku[(0,4)])
5
>>> print(sudoku[(2,2)])
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    print sudoku[(2,2)]
KeyError: (2, 2)
```

Chaque clé dans un dictionnaire est unique.

- Les dictionnaires permettent tous les opérateurs de type standard mais n'autorisent pas d'opérations telles que la concaténation, la répétition ou l'extraction d'un groupe d'éléments contigus car les dictionnaires ne sont pas des séquences, c'est-à-dire les éléments ne sont pas disposés dans un ordre particulier.

Les dictionnaires

- Contrairement à une liste, nous ne faisons pas appel à une méthode particulière telle que `append()` pour ajouter un nouvel élément à un dictionnaire. Il suffit de créer une nouvelle paire clé-valeur.

```
>>> print(traduction)
{'mouse': 'souris', 'keyboard': 'clavier'}
>>> traduction["computer"] = "ordinateur"
>>> print(traduction)
{'computer': 'ordinateur', 'mouse': 'souris', 'keyboard': 'clavier'}
```

dict() : renvoie un dictionnaire issu par exemple d'une séquence où ses éléments doivent être associés deux à deux. Le premier élément constituera une nouvelle clé et le second sa valeur associée.

```
>>> dict([[1,2], [6,7], [3,9]])
{1: 2, 3: 9, 6: 7}
>>> dict([(1,2), (6,7), (3,9)])
{1: 2, 3: 9, 6: 7}
>>> dict(((1,2), (6,7), (3,9))))
{1: 2, 3: 9, 6: 7}
```

Supprimer des éléments d'un dictionnaire ou des dictionnaires entiers

- Pour enlever un élément à un dictionnaire, on utilise l'instruction `del` et pour connaître le nombre d'éléments du dictionnaire, on se sert de la fonction `len`.

```
>>> del(traduction["mouse"])
>>> print (len(traduction))
2
>>> print(traduction)
{'computer': 'ordinateur', 'keyboard': 'clavier'}
>>> traduction.pop('computer')
'ordinateur'
>>> traduction.clear()
>>> print(traduction)
{}
>>> del(traduction)
>>> print(traduction)
```

Supprime et retourne l'entrée.

Supprime toutes les entrées d'un dictionnaire.

Supprime un dictionnaire.

```
Traceback (most recent call last):
  File "<pyshell#44>", line 1, in <module>
    print traduction
NameError: name 'traduction' is not defined
```

Accès aux valeurs d'un dictionnaire

`keys()` : renvoie la liste des clés utilisées dans le dictionnaire.

Cela permet de parcourir un dictionnaire et d'accéder à ses valeurs.

```
>>> dictionnaire = {'rouge': 'red', 'vert': 'green', 'bleu': 'blue'}
>>> for cle in dictionnaire.keys():
    print (cle, « --- » ,dictionnaire[cle])

bleu---blue
vert---green
rouge---red
```

Depuis Python 2.2, ce n'est plus nécessaire d'utiliser la méthode `keys()`.

```
>>> dictionnaire = {'rouge': 'red', 'vert': 'green', 'bleu': 'blue'}
>>> for cle in dictionnaire:
    print (cle, « --- » ,dictionnaire[cle])

bleu---blue
vert---green
rouge---red
```

On doit vérifier si un dictionnaire possède une clé avant de tenter d'accéder à sa valeur.

```
>>> if 'noir' in dictionnaire: dictionnaire['noir']  
>>> 'rouge' in dictionnaire  
True
```

Comment modifier un dictionnaire ?

Vous pouvez mettre à jour un dictionnaire en lui ajoutant une nouvelle entrée ou en modifiant une entrée existante.

```
>>> dictionnaire = {'rouge': 'red', 'vert': 'green', 'bleu': 'blue'}  
>>> dictionnaire['noir'] = 'black'  
>>> dictionnaire['bleu'] = 'BLUE'  
>>> print(dictionnaire)  
{'bleu': 'BLUE', 'noir': 'black', 'vert': 'green', 'rouge': 'red'}
```

Lorsqu'on affecte une valeur à une clé qui n'existe pas encore, la clé est créée dans le dictionnaire et la valeur est ajoutée. Mais si la clé existe déjà, alors la valeur existante est remplacée par la nouvelle.

```
>>> dictionnaire = {1: "vert", 1:"VERT"}  
>>> print(dictionnaire)  
{1: 'VERT'}
```

Fonctions et méthodes permettant de manipuler des dictionnaires.

D.values() : renvoie la liste des valeurs utilisées dans le dictionnaire D.

D.items() : renvoie une liste équivalente de tuples à partir d'un dictionnaire D.

```
>>> print(traduction.keys())  
['computer', 'keyboard']  
>>> print(traduction.values())  
['ordinateur', 'clavier']  
>>> print(traduction.has_key("mouse"))  
False  
>>> print(traduction.items())  
[('computer', 'ordinateur'), ('keyboard', 'clavier')]
```

D.copy() : effectue une vraie copie d'un dictionnaire D au lieu d'un alias.

```
>>> conversion = traduction.copy()  
>>> conversion["mouse"] = "souris"  
>>> print traduction  
{'computer': 'ordinateur', 'keyboard': 'clavier'}  
>>> print conversion  
{'computer': 'ordinateur', 'mouse': 'souris', 'keyboard': 'clavier'}
```

```
L=[["Zineb",17],["Salma",16.75],["Amine",15]]
```

```
def trouverNote(M,nom):  
    for i in range(len(M)):  
        if M[i][0]==nom:  
            return M[i][1]  
    return -1
```

Exercice :

On dispose d'un dictionnaire associant à des noms de commerciaux d'une société le nombre de ventes qu'ils ont réalisées.

Par exemple :

ventes= {"**Amine**":14, "**Rachid**":19, "**Hamza**":15, "**Taha**":21, "**Aya**":17}

- 1) Écrire une fonction qui prend en entrée un dictionnaire et renvoie le nombre total de ventes dans la société.
- 2) Écrire une fonction qui prend en entrée un tel dictionnaire et renvoie le nom du vendeur ayant réalisé le maximum de ventes. Si plusieurs vendeurs sont *maximaux* sur ce critère, la fonction devra retourner le nom de l'un d'entre eux.

Exercice :

Soit L une liste de nombres entiers et codesH un dictionnaire qui associe à chaque entier de cette liste une chaîne de caractères binaire (0 ou 1).

Ecrire une fonction `def compresse(L, codesH)` qui construit une chaîne binaire en remplaçant chaque entier de L par sa valeur codée dans codesH.

Par exemple si:

`L=[12,29,46,29,31,8,12,55,29,8,12,29,31,29,8,29,8]`

`codesH={55: '1011', 8: '01', 12: '00', 29: '11', 46: '1010', 31: '100'}`

Alors:

`compresse(L, codesH)` donne ➔

`'0011101011100010010111101001110011011101'`

Exercice (suite):

Programmer la fonction `decompresse(codesH,B)` qui permet de retrouver les entiers à partir du dictionnaire et de la chaîne binaire.

Par exemple si

`codesH={55: '1011', 8: '01', 12: '00', 29: '11', 46: '1010', 31: '100'}`

`B='0011101011100010010111101001110011011101'`

Alors `decompresse(codesH,B)` donne ➔

`[12, 29, 46, 29, 31, 8, 12, 55, 29, 8, 12, 29, 31, 29, 8, 29, 8]`